# FIKS: Turning Silent Tool Failures into Gated Repair Pull Requests for MCP Servers

System Design, Pilot Evaluation, and a Pre-Registered Protocol

[TODO: author names and affiliations]

Repository owner: `github.com/vats98754`

Preprint, October 2026

**Abstract**

Tool-using agents treat tool results as ground truth [39], yet MCP servers can return well-formed *successful* responses whose content is wrong [33], and agents then build on the bad value [41]. Runtime monitors can warn the current agent [34], but the tool stays broken for every later agent, and spec-diff tools are blind when an upstream API changes behaviour without changing its schema [32]. We present FIKS, which closes this loop for MCP servers. A proxy checks every successful tool result against an *outcome contract* (ranges, nullness, counts, freshness, agreement with the raw upstream HTTP exchange, read-after-write), and returns a receipt to the agent. On a violation it captures the exact call and upstream exchange, compiles it into a deterministic reproduction test that must fail on the current code *for the named property*, and asks a coding agent to repair the server. FIKS opens a pull request only after that test, the full contract suite, every stored trace, typecheck, lint and CI on the exact commit all pass, and it never merges. In a pilot on one mock vendor with 5 seeded behavioural drifts whose OpenAPI document is unchanged, FIKS flagged all 5 drifts, produced a valid fail-first repro for each, and obtained a green-CI pull request in one repair attempt per drift (1.4–1.6 minutes from captured violation to PR; \$0.11–\$0.21 API-price estimate per repair). In a 6-task, 5-trial agent bench, pass$^5$ was 50.0% without FIKS, 83.3% with receipts and 100.0% after the fixes were merged; with only 2 discordant tasks this difference is not statistically significant (two-sided sign test $p = 0.50$), and an earlier receipt wording performed *worse* than no monitor (33.3% vs 66.7%). From a review of 508 verified sources we argue that the defensible novelty is the combination of trigger, artifact and gate placement, not any component. We release the full pre-registered protocol for a confirmatory study (four hypotheses, Holm-corrected) and state clearly that its experiments have not yet been run.

## 1 Introduction

Agents act on what tools tell them. Anthropic describes agents as systems that must obtain "ground truth" from the environment at each step [39]; in the Model Context Protocol (MCP) that ground truth arrives as tool results. When a tool fails loudly, with an exception or an error result, existing machinery handles it: the agent retries, the error tracker files an issue, and coding agents can be pointed at the stack trace [13, 40]. The harder case is a tool that fails *silently*. A study of 833 MCP runtime-fault threads found servers that return well-formed success responses despite incorrect behaviour [33]. Agents given such responses fabricate or adopt wrong values [41], LLM judges struggle to detect false success from transcripts [2], and even the tools inside agent benchmarks report writes that never happened [5].

A common source of silent failure is upstream drift: the vendor API behind an MCP server changes behaviour (a unit, a field name, pagination, a timestamp format, a write that now needs a header) while its published schema stays the same. Spec-diff tools compare documents and therefore report nothing [32]; in library ecosystems a large share of documented breaking changes are behavioural rather than signature changes [26]. Runtime outcome monitors close part of the gap: they check results and tell the running agent [31, 34]. But detection alone leaves the tool broken for every later session, model and user.

FIKS turns the runtime violation into a durable fix. Its premise is that a violation observed in live traffic is already most of a reproduction: the exact arguments, the raw upstream exchange and the property that failed.

The reproduction literature identifies a valid failing test as the bottleneck of automated repair [3, 20, 30], and warns that fail-to-pass tests can still be lax [42] or admit partial patches [19]. FIKS therefore (i) compiles the trace into a test that must fail on the current code *for the stated reason*, (ii) gives that test to a sandboxed coding agent, and (iii) places every gate, including CI on the exact commit, *before* a pull request is opened.

**Contributions.**

1. **A loop and its gate placement** (§3): per-call outcome contracts on successful MCP results, an immediate receipt to the agent, trace capture with the raw upstream exchange, a named-property fail-first reproduction, gated repair, and a pull request that is never merged automatically.
2. **An implementation and a pilot evaluation** (§4) on five seeded drifts, with every number produced by a script from committed logs (Tables 3–6), including a negative result on receipt wording.
3. **A positioning against 508 verified sources** (§5), with an explicit list of what is *not* new.
4. **A pre-registered confirmatory protocol** (§6) whose design, power analysis and analysis plan were fixed before any confirmatory run.

**Scope.** This paper reports a pilot. Its evidence comes from one mock vendor, one MCP server, drifts and contracts written by the same authors, six agent tasks and one model family. We say in §7 what the pilot cannot show, and the protocol in §6 is designed to test it.

# 2 Problem: silent failures at the agent–tool boundary

Let a tool $\tau$ map arguments $x$ to a result $r = \tau(x)$ by calling an upstream service $U$; the call produces an upstream exchange $e = (\text{req}, \text{resp})$. A *silent failure* is a call where $r$ is a well-formed success (no exception, no `isError`, valid against the declared `outputSchema`) but $r$ misstates the state of $U$. The MCP specification lets a tool declare an `outputSchema` for its structured content, with output validation left to clients [37]; a schema can check shape, not whether a price is in dollars or cents.

We focus on *upstream behavioural drift*: $U$ changes at time $t_0$ while its published specification and $\tau$'s code do not. From $t_0$ on, every call that touches the changed behaviour is silently wrong, for every agent, until $\tau$ is fixed. Three properties make this setting tractable for a closed loop: (a) the violation is observable per call, because the proxy sees both $r$ and $e$; (b) the trace $(x, e, r, \text{property})$ is a deterministic reproduction once $e$ is replayed; (c) the fix belongs to $\tau$'s repository, so it can be proposed as a pull request and reviewed.

# 3 Design of fiks

## 3.1 Outcome contracts and receipts

A contract is a per-tool list of invariants evaluated on each successful result (Table 1). Most of the vocabulary is standard in data validation [8]; two kinds use the proxy's position between tool and upstream. `equalsUpstream` compares a result field with the field the upstream actually sent in the same call, and `readAfterWrite` re-reads a created resource from the upstream to confirm a write persisted. Each verdict is *verified*, *violated* or *unverifiable*. The proxy attaches a receipt to the result in three places: MCP `_meta`, a final text block, and a field in `structuredContent`, because some clients show the model only the structured content. A receipt states the violated property, the upstream facts the proxy observed, and what the agent should not do with the value. The receipt for the pilot's renamed-price drift reads:

```
[fiks] VIOLATED get_product: price is null, but p-1001 should have one; vendor sent unit_amount = 1199 (in
    20-2000 USD). Don't say there is no price; if you use the vendor value, say unconfirmed.
```

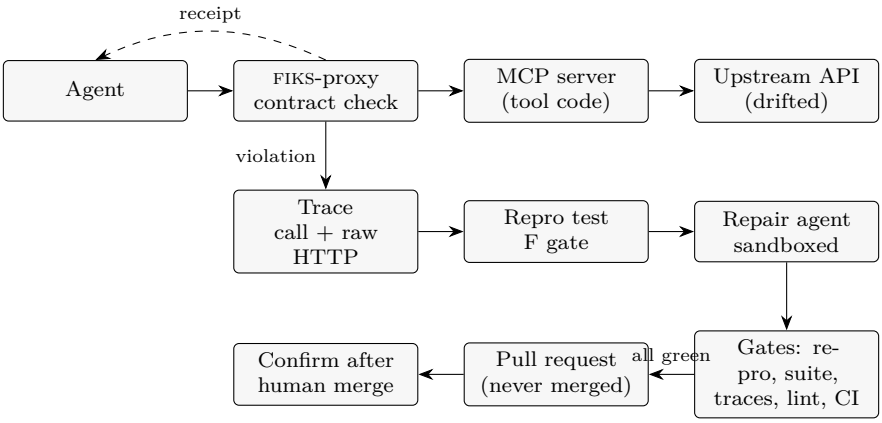



Figure 1: The FIKS loop. The receipt reaches the running agent immediately; the fix reaches every later agent once a human merges the pull request.

Table 1: Contract vocabulary used in the pilot. The first five kinds are standard data-quality checks; the last two rely on seeing the upstream exchange of the same call.

| Kind | Checks |
|---|---|
| `range` | numeric field within [min, max] in a stated unit |
| `notNull`, `nonEmpty` | field present / list non-empty for known-good inputs |
| `unique` | no duplicate ids in a list |
| `countMatchesTotal` | returned items equal the reported total, or a next-page signal exists |
| `freshness` | timestamp within a plausible window of the call time |
| `equalsUpstream` | field equals the upstream response's field (JSON or instant) |
| `readAfterWrite` | a created resource can be read back from the upstream |

### 3.2 Trace, reproduction and the F gate

On a violation the proxy stores a trace: tool, arguments, the raw upstream requests and responses recorded by an HTTP tap, the contract and the violated property ids. FIKS generates a test in the server's own repository that replays the recorded upstream exchange through the server's real code and asserts the contract. The **F gate** accepts the test only if it fails on the current code on each of three runs, and the first failure line names exactly the target property ids; a test that fails for another reason, or passes, is rejected. This makes "fails for the stated reason" a deterministic check that needs no reference patch, unlike LLM-judged reproduction [3, 30].

### 3.3 Repair and gates

A coding agent (Claude Agent SDK; claude-opus-5-5 in the pilot) receives the violation, the trace and the failing test in an isolated git worktree. A tool-permission callback lets it write only under `src/`, run only the project's typecheck, lint and test commands, and read only inside the worktree; tests, fixtures and traces are hashed at the snapshot, and any change to them fails the run. When a correct fix changes upstream requests (for example, a new required header), FIKS itself re-records the affected fixtures from the committed recordings with a frozen clock, in a separate commit containing only re-recordings. The **P gate** then requires the repro to pass, together with the full contract suite, every stored trace, typecheck and lint. FIKS pushes the branch, waits for CI on that exact commit, and opens a pull request only if CI is green; if the pull request's head differs from the commit that passed CI, it is closed. The pull request body states what broke, the trace, the test, the diff, before/after results and the gate outcomes, and says that a green suite is not a proof of correctness. FIKS never merges. After a human merge, `fiks confirm` re-runs each repaired call live and replays every stored trace.

Table 2: Seeded behavioural drifts (vendor v2). The OpenAPI document does not change.

| Drift | Upstream change | Silent symptom in the unfixed tool |
|---|---|---|
| D1 unit | prices become integer cents, same field name | prices 100× too high |
| D2 rename | `price` renamed to `unit_amount` | price returned as `null` |
| D3 lost write | order persists only with a new required header | create succeeds; order cannot be read back |
| D4 pagination | search results paginated | only page 1 returned; count < total |
| D5 timestamp | ISO-8601 becomes epoch milliseconds | dates parsed as 1970 |

# 4 Pilot evaluation

## 4.1 Setup

**Substrate.** A mock vendor API with a version switch (v1, v2) and an MCP server (`shop-mcp`, four tools: `search_products`, `get_product`, `create_order`, `get_order`) in its own repository, so pull requests and CI are real. The vendor's OpenAPI document is identical in v1 and v2. Version v2 introduces five drifts (Table 2). We wrote the drifts and the contracts; §7 discusses the circularity this creates.

**Agent bench.** 6 tasks (T1–T5 each exposed to one drift; T6 a control), 5 trials each, agent claude-sonnet-5-5 via the Claude Agent SDK with only the shop MCP server and a `submit_answer` tool. A deterministic grader compares the submitted answer with the vendor's ground truth; a hedged wrong value counts as wrong, and an answer never submitted counts as a failure (reported separately as "no answer"). Phases: before drift through FIKS; after drift without FIKS (agent talks to the server directly); after drift with FIKS receipts; after drift with the five FIKS pull requests merged. pass$^k$ uses the unbiased $\binom{c}{k}/\binom{n}{k}$ estimator averaged over tasks [10, 49]; intervals are Wilson [9], computed over trials and therefore ignoring clustering by task, which makes them too narrow [7].

**Provenance.** All numbers in this section are emitted by `paper/scripts/pilot_numbers.py` from committed logs (`bench/results/`, repair records, and a snapshot of the pull requests); `-check` rebuilds them and fails on any difference. Bench runs used FIKS commit `705e989` and the server at tag `seed-v1`.

## 4.2 Detection and false alarms

In the drift phase, 41 of 47 tool calls received a violated receipt, covering the violated properties of all five drifts (D1 `items.price.range`, D2 `price.notNull`, D3 `readAfterWrite`, D4 `countMatchesTotal`, D5 `created_at` freshness and upstream agreement). On clean v1 traffic, 0 of 37 calls were flagged (Wilson 95% upper bound 9.4% per call; 11.4% per trial over 30 trials). These bounds are weak: the pilot's clean traffic is small and comes from the same six tasks the contracts were written against.

## 4.3 The repair loop

Table 3 reports the loop for each drift. All 5 reproductions passed the F gate, all 5 repairs succeeded in one agent attempt (10–14 turns, median 12), and all 5 pull requests had green CI before opening. Total repair cost was $0.77 at API list prices; the runs used a subscription token, so this is an estimate, not a bill. After the repository owner merged the five pull requests, `fiks confirm` found each drift resolved (5/5). Across the pilot and later demonstration runs, FIKS opened 10 repair pull requests, all with green checks; 5 were merged, 4 demonstration pull requests target throw-away branches and remain open, and 1 was a duplicate opened by an operator error and closed. The merges were performed by the authors' assistant at the owner's request and were not reviewed line by line; they are not evidence of maintainer acceptance [16].

## 4.4 Agent outcomes

Table 4 gives pass$^k$ per phase and Table 5 the per-task successes. Without FIKS, pass$^5$ fell from 100.0% to 50.0%; with receipts it was 83.3%; with the fixes merged it returned to 100.0% with no receipts needed. Three

Table 3: Repair loop per drift. "Min. to PR" runs from the captured violation to the opened pull request, including the CI wait. Cost is the Agent SDK's API-price estimate.

| Drift | Violated property | F gate | Att. | Turns | Cost ($) | Min. to PR | PR | Merged | Confirmed |
|---|---|---|---|---|---|---|---|---|---|
| D1 | `items.price.range` | yes | 1 | 13 | 0.11 | 1.4 | #2 | yes | yes |
| D2 | `price.notNull` | yes | 1 | 10 | 0.14 | 1.4 | #1 | yes | yes |
| D3 | `readAfterWrite` | yes | 1 | 14 | 0.21 | 1.6 | #3 | yes | yes |
| D4 | `countMatchesTotal` | yes | 1 | 11 | 0.19 | 1.5 | #7 | yes | yes |
| D5 | `created_at.freshness, created_at.equalsUpstream` | yes | 1 | 12 | 0.12 | 1.4 | #4 | yes | yes |

Table 4: Agent bench (claude-sonnet-5-5, 6 tasks × 5 trials per phase). pass[1] CI is Wilson over trials and ignores clustering by task.

| Phase | pass[k] (%), k=1..5 | | | | | pass[1] 95% CI | Corr. | Wrong | Decl. | No ans. | Viol. calls |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Before drift (v1), through fiks | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | 89–100 | 30 | 0 | 0 | 0 | 0 |
| After drift (v2), no fiks | 60.0 | 55.0 | 51.7 | 50.0 | 50.0 | 42–75 | 18 | 5 | 5 | 2 | 0 |
| After drift (v2), fiks receipts | 83.3 | 83.3 | 83.3 | 83.3 | 83.3 | 66–93 | 25 | 0 | 5 | 0 | 41 |
| After drift (v2), fiks fixes merged | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | 89–100 | 30 | 0 | 0 | 0 | 0 |

observations temper this. First, only 2 tasks differ in the receipts-vs-no-FIKS comparison (0 in the other direction), so a task-level paired sign test gives $p = 0.50$; the pilot cannot establish an effect. Second, all 5 wrong answers without FIKS came from one task (T5, the timestamp drift: the agent reported 1970), and 2 further failures were trials where the agent stated the correct price but never submitted it; counting those as correct would raise the no-FIKS pass[5] to 66.7%, about half the gap. Third, the 5 declines with receipts are all the lost-write task T3, where the vendor really did not persist the order; "could not confirm" is arguably the right answer there, the grader counts it as a failure, and the same 5 declines occur without FIKS.

### 4.5 A negative result: receipt wording

Receipts are part of the method. Our first wording (v1) stated the violation and told the agent not to quote the value; it carried no observed upstream facts. With it, pass[5] was 33.3% against 66.7% without FIKS on the same day (Table 6): the agent declined 15 times rather than answer. The second wording (v2) adds the upstream value the tool dropped, labelled unconfirmed, and next steps. We changed the wording after seeing these results, so the v2 numbers are post hoc and need the independent test in §6.

## 5 Related work and novelty boundary

**Silent failures and agent benchmarks.** MCP fault studies classify silent faults from issue threads but do not reproduce or repair them [33]. Controlled injection shows agents adopt plausible wrong values [41], and false success is hard to detect from transcripts [2]. Agent benchmarks such as $\tau$-bench assume tools are correct [49], an assumption an executable-contract audit found violated [5].

**Runtime monitors.** Outcome Monitors check results and give the agent recovery affordances, explicitly without repairing the tool [34]; verimcp re-derives outcomes after git, file and CI calls [31]; postcondition checks at the agent–tool boundary reduce duplicate actions [29]. Action-side enforcement such as AgentSpec checks planned actions rather than results [44]. FIKS's receipt step belongs to this line and is not new on its own.

**Specification, contract and data validation.** Spec-diff tools find nothing when the document does not change [32]; consumer-driven contracts need the provider to run consumer pacts [18]; schema-derived fuzzing finds schema violations [50]. Large-scale studies count breaking changes in library APIs [26, 45]. Data validation systems enforce the same invariant vocabulary on batches [8], and invariant inference goes back to Daikon [17]. Record/replay tools turn captured traffic into regression tests whose oracle is the old response [25]; that oracle encodes the bug when the recording is made after the drift.

Table 5: Per-task successes out of 5. T1–T5 are each exposed to one drift (D2, D1, D3, D4, D5); T6 is a control.

| Phase | T1 | T2 | T3 | T4 | T5 | T6 |
|---|---|---|---|---|---|---|
| Before drift (v1), through fiks | 5/5 | 5/5 | 5/5 | 5/5 | 5/5 | 5/5 |
| After drift (v2), no fiks | 3/5 | 5/5 | 0/5 | 5/5 | 0/5 | 5/5 |
| After drift (v2), fiks receipts | 5/5 | 5/5 | 0/5 | 5/5 | 5/5 | 5/5 |
| After drift (v2), fiks fixes merged | 5/5 | 5/5 | 5/5 | 5/5 | 5/5 | 5/5 |

Table 6: Receipt wording history on the drifted vendor (same tasks and model). v1 lowered success below no monitor.

| Arm (drifted vendor) | pass[1] | pass[5] | Wrong | Declined | No answer |
|---|---|---|---|---|---|
| No fiks (2026-10-18 run) | 66.7 | 66.7 | 5 | 5 | 0 |
| fiks receipts, wording v1 | 43.3 | 33.3 | 0 | 15 | 2 |
| No fiks (2026-10-20 run) | 60.0 | 50.0 | 5 | 5 | 2 |
| fiks receipts, wording v2 | 83.3 | 83.3 | 0 | 5 | 0 |

**Reproduction and repair.** Issue-to-test generation is hard and its validity is judged by fail-to-pass or by LLMs [3, 24, 30]; a true reproduction is the bottleneck of test-driven repair [20], yet fail-to-pass tests can be lax [42] or admit partial patches [19], and gate-rejected fixes often regress protected cases [43]. Agentic repair scaffolds [46, 47] and dependency-break repair [38, 48] start from issues, compile errors or failing tests. Structured upstream evidence helps dependency repair while raw evidence hurts [48], which motivates compiling the trace into a test rather than pasting it.

**Production-driven and commercial loops.** ClearView patched deployed software from learned invariants [35]; Itzal validated production-driven patches on traffic [15]; SapFix shipped detect–test–repair with human approval [22]. In 2026, observability and alerting products open fix pull requests from errors, incidents or LLM scans and iterate CI after opening [13, 27, 36, 40], and some measure recurrence after fixes [12]. AegisFlow detects silent pipeline drift statistically and deploys validated patches autonomously [4]. A granted patent covers failure-triggered LLM repair with a pull request accompanied by a test [1].

**What is and is not new.** Table 7 compares the closest systems by loop step, from their documentation or papers. Each step exists somewhere. In the sources we reviewed (searched up to 2026-10-21; patent databases only partly searched), we found no system that combines: a trigger that is a deterministic contract violation in a *successful* tool result, checked per call against the raw upstream exchange; a committed test that must fail first for the named property; and all gates, including CI on the exact commit, before the pull request opens, with a receipt to the agent while the fix is pending. We claim only that combination, and the pilot does not yet show that it outperforms a composition of existing parts; that is the question of §6.

# 6 Pre-registered confirmatory protocol

The full pre-registration is `paper/PLAN.md` in the repository; build-phase amendments made before any freeze are logged in its §0.B. It was produced by competing designs, a judge panel and repeated adversarial critique, and its power numbers are recomputed by script. No confirmatory run has been performed. The protocol compares FIKS with two strong alternatives built on the same monitor, model, budget and sandbox: **L1**, the same agent and gates with a pre-PR replay of the triggering trace but no visible reproduction test; and **COMP**, the strongest documented composition, monitor → alert → coding agent → PR → CI with source-run replay. Four hypotheses form the confirmatory family (Holm, FWER 0.05):

**H1 (trigger).** On a pre-registered frame of at least 1,500 blind-authored catalog entities over two domains, fewer than 2% of entities give rise to any false positive (exact binomial test), with fewer than 1% of inputs flagged.

Table 7: Loop steps covered by the closest systems (Y documented, P partial or different form, N not done, ? not described). Sem: semantic check of a successful result; Rcpt: verdict to the agent; Trace: raw upstream captured; F: test must fail first for the stated reason; Gates: suite and CI pass before the PR.

| System | Sem | Rcpt | Trace | Repro test | F | Fix | Gates | PR | Confirm |
|---|---|---|---|---|---|---|---|---|---|
| FIKS (this paper) | Y | Y | Y | Y | Y | Y | Y | Y | Y |
| Outcome Monitors [34] | Y | Y | N | N | N | N | N | N | N |
| verimcp [31] | Y | Y | P | N | N | N | N | N | N |
| AegisFlow [4] | Y | N | P | N | N | Y | P | N | Y |
| LangSmith Engine [27] | P | N | P | N | P | Y | N | Y | Y |
| Cleric [12] | P | N | P | N | N | Y | ? | Y | Y |
| PostHog Self-driving [36] | N | N | P | Y | ? | Y | N | Y | ? |
| Sentry Seer [40] | N | N | P | P | N | Y | N | Y | N |
| Keploy [25] | N | N | Y | Y | N | N | N | N | N |
| SapFix [22] | N | N | P | Y | P | Y | P | Y | N |
| Itzal [15] | P | N | P | N | P | Y | P | P | N |

**H2 (artifact).** Over a fixed, arm-independent patch pool graded by a held-out oracle, the trace-derived named-property repro has higher balanced accuracy than a fail-first repro written by a coding agent from the same alert, raw trace and replay helper.

**H3 (visible repro).** FIKS exposes fewer green-but-incorrect pull requests than L1, with a correct-PR loss no larger than 0.10 (intersection–union test with a direction guard).

**H4 (vs composition).** The same comparison against COMP.

Drifts are authored blind to the contracts by a separate process and applied by a third-party response-rewriting injector; benign upstream evolution, mutation operators frozen before contracts, four real silent-failure bugs in public MCP servers reproduced in private clones, and an adversarial-trace security suite are exploratory arms. Expected losses are pre-registered as well: out-of-vocabulary string corruption, legitimate evolution that a fixed contract misreads as a fault, and cases where issue text already explains the bug.

# 7 Threats to validity and limitations

**Circularity.** The same authors wrote the drifts, the contracts and the tasks; detection of D1–D5 is therefore close to a construction property. Mutation-based oracle assessment and real faults are needed to measure what contracts miss [21, 23], and runtime contract vocabularies have known blind spots on out-of-vocabulary faults [34].

**Scale and statistics.** Six tasks and five trials cannot support task-level significance; trial-level intervals ignore clustering [7]. The false-alarm bound comes from 37 clean calls.

**Partial or overfitted patches.** A green suite shows the repro and the recorded contracts pass; it does not prove correctness [19, 42]. The pilot measured no held-out oracle.

**Security.** The repair agent reads attacker-controllable vendor bytes and its code runs in tests. Restricting tools does not stop injected agents from writing malicious source [28], test-passing patches can hide vulnerabilities [11], and injection through tool output is measurable [6, 14]. In the pilot the agent ran without an OS or network sandbox; a container backend and an injection suite are part of the protocol, not of these results.

**Legitimate evolution.** A fixed contract cannot tell a vendor fault from an intended change; the pilot always proposed a code fix. A contract-update outcome is designed but unevaluated.

**External validity.** One mock vendor, one server, one model family (Claude) for both bench and repair, and seeded rather than organic drift. Merged pull requests were merged at the owner's request without line-by-line review, so they say nothing about maintainer acceptance.

## 8 AI use and ethics statement

The system, the experiments and this manuscript were produced with substantial assistance from Claude (Claude Code and the Claude Agent SDK; claude-opus-5-5 and claude-sonnet-5-5), directed by the authors. Literature search and source verification were done by agents that recorded the supporting passage for each citation; every number in §4 is generated from committed logs by a script. The authors are responsible for all claims. No third-party repository was written to: pull requests were opened only on repositories owned by the authors, and real-bug reproductions use private clones.

## 9 Conclusion

FIKS shows that a silent tool failure observed in live agent traffic can be turned, within minutes, into a named-property failing test and a gate-passing pull request against the tool. The pilot demonstrates feasibility on seeded drifts; it does not show that the loop beats a composition of existing monitors and coding agents, that its contracts generalize beyond the drifts they were written for, or that its pull requests would be accepted. The pre-registered protocol is designed to answer exactly those questions.

## References


[1] Rajeev Prabhu Acharya, Siamak Ahari, and LLC) Vinay Rao (assignee Microsoft Technology Licensing. Detecting and repairing reliability issues in operating systems and applications using a generative artificial intelligence system (us 12,711,009 b2). USPTO (granted 18 Aug 2026; pre-grant pub. US 2025/0045148 A1; provisional 31 Jul 2023), 2026. URL https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/12711009. Accessed 2026-10-21.

[2] Laksh Advani. From confident closing to silent failure: Characterizing false success in llm agents. arXiv:2606.09863, 2026. URL https://arxiv.org/abs/2606.09863. Accessed 2026-10-21.

[3] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. Otter: Generating tests from issues to validate swe patches. arXiv:2502.05368, 2025. URL https://arxiv.org/abs/2502.05368. Accessed 2026-10-21.

[4] Muhammad Bilal Awan, Zubair Hussain, and Abdul Shahid. Aegisflow: A multi-agent agentic ai framework for autonomous remediation and self-healing in fragile data ecosystems. arXiv:2610.06971, 2026. URL https://arxiv.org/abs/2610.06971. Accessed 2026-10-21.

[5] Rohith Reddy Bellibatlu, Zichong Wang, and Wenbin Zhang. Do agent benchmarks do what they say? an executable-contract audit of tool-using agent environments. arXiv:2609.37315, 2026. URL https://arxiv.org/abs/2609.37315. Accessed 2026-10-21.

[6] Dimitrios Stamatios Bouras, Yihan Dai, and Sergey Mechtaev. Authority is not a string: A capability-scoped harness for prompt-injection-resistant coding agents. arXiv:2609.08371, 2026. URL https://arxiv.org/abs/2609.08371. Accessed 2026-10-21.

[7] Sam Bowyer, Laurence Aitchison, and Desi R. Ivanova. Position: Don't use the clt in llm evals with fewer than a few hundred datapoints. arXiv:2503.01747, 2025. URL https://arxiv.org/abs/2503.01747. Accessed 2026-10-21.

[8] Eric Breck, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data validation for machine learning. SysML 2019 (Google Research), 2019. URL https://research.google/pubs/data-validation-for-machine-learning/. Accessed 2026-10-21.

[9] Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. Statistical Science 16(2):101-133, 2001. URL https://doi.org/10.1214/ss/1009213286. Accessed 2026-10-21.

[10] Mark Chen, Jerry Tworek, Heewoo Jun, and (58 authors). Evaluating large language models trained on code. arXiv:2107.03374, 2021. URL https://arxiv.org/abs/2107.03374. Accessed 2026-10-21.

[11] Simin Chen, Yixin He, Suman Jana, and Baishakhi Ray. Red teaming program repair agents: When correct patches can hide vulnerabilities. arXiv:2509.25894, 2025. URL https://arxiv.org/abs/2509.25894. Accessed 2026-10-21.

[12] Peter Richens (Cleric). How we verify cleric's production fixes. cleric.ai blog (2026-07-22), 2026. URL https://cleric.ai/blog/verifying-fixes. Accessed 2026-10-21.

[13] Cognition. Devin oncall. docs.devin.ai, 2026. URL https://docs.devin.ai/product-guides/devin-oncall. Accessed 2026-10-21.

[14] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. arXiv:2406.13352, 2024. URL https://arxiv.org/abs/2406.13352. Accessed 2026-10-21.

[15] Thomas Durieux, Youssef Hamadi, and Martin Monperrus. Production-driven patch generation and validation. arXiv:1609.06848, 2016 (v2 2018). URL https://arxiv.org/abs/1609.06848. Accessed 2026-10-21.

[16] Ramtin Ehsani, Sakshi Pathak, Shriya Rawal, Abdullah Al Mujahid, Mia Mohammad Imran, and Preetha Chatterjee. Where do ai coding agents fail? an empirical study of failed agentic pull requests in github. arXiv:2601.15195, 2026. URL https://arxiv.org/abs/2601.15195. Accessed 2026-10-21.

[17] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The daikon system for dynamic detection of likely invariants. Science of Computer Programming 69(1-3):35-45, Dec 2007, 2007. URL https://homes.cs.washington.edu/~mernst/pubs/daikon-tool-scp2007-abstract.html. Accessed 2026-10-21.

[18] Pact Foundation. How pact works. docs.pact.io, 2026. URL https://docs.pact.io/getting_started/how_pact_works. Accessed 2026-10-21.

[19] Yaoqi Guo, Yang Liu, Jie M. Zhang, Yun Ma, Yiling Lou, and Zhenpeng Chen. Swe-doctor: Guiding software engineering agents with runtime diagnosis from multi-faceted bug reproduction tests. arXiv:2607.00990, 2026. URL https://arxiv.org/abs/2607.00990. Accessed 2026-10-21.

[20] Kevin Han, Siddharth Maddikayala, Tim Knappe, Om Patel, Austen Liao, and Amir Barati Farimani. Tdflow: Agentic workflows for test driven development. arXiv:2510.23761, 2025. URL https://arxiv.org/abs/2510.23761. Accessed 2026-10-21.

[21] Gunel Jahangirova, David Clark, Mark Harman, and Paolo Tonella. Test oracle assessment and improvement. ISSTA 2016, pp. 247-258, 2016. URL https://discovery.ucl.ac.uk/1493269/1/main.pdf. Accessed 2026-10-21.

[22] Yue Jia, Ke Mao, and Mark Harman. Finding and fixing software bugs automatically with sapfix and sapienz (meta engineering blog). Meta Engineering blog, 13 Sep 2018 (paper: Marginean et al., ICSE-SEIP 2019 [verify paper text]), 2018. URL https://engineering.fb.com/2018/09/13/developer-tools/finding-and-fixing-software-bugs-automatically-with-sapfix-and-sapienz/. Accessed 2026-10-21.

[23] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? FSE 2014, pp. 654-665, 2014. URL https://homes.cs.washington.edu/~rjust/publ/mutants_real_faults_fse_2014.pdf. Accessed 2026-10-21.

[24] Sungmin Kang, Juyeon Yoon, and Shin Yoo. Large language models are few-shot testers: Exploring llm-based general bug reproduction. arXiv:2209.11515, 2023. URL https://arxiv.org/abs/2209.11515. Accessed 2026-10-21.

[25] Keploy. What is keploy? Keploy Docs; GitHub keploy/keploy (Apache-2.0), 2026. URL https://keploy.io/docs/concepts/what-is-keploy. Accessed 2026-10-21.

[26] Dezhen Kong, Jiakun Liu, Lingfeng Bao, and David Lo. Towards better comprehension of breaking changes in the npm ecosystem. arXiv:2408.14431, 2025. URL https://arxiv.org/html/2408.14431v2. Accessed 2026-10-21.

[27] LangChain. Langsmith cloud changelog (engine entries, sep-oct 2026). LangChain docs, 2026. URL https://docs.langchain.com/langsmith/changelog. Accessed 2026-10-21.

[28] Liu, Zhao, Lyu, Zhang, Wang, and Lo. "your ai, my shell": Demystifying prompt injection attacks on agentic ai coding editors. arXiv:2509.22040, 2025. URL https://arxiv.org/abs/2509.22040. Accessed 2026-10-21.

[29] Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. Verified tool calls improve llm agent reliability under non-atomic failures. arXiv:2608.02645, 2026. URL https://arxiv.org/abs/2608.02645. Accessed 2026-10-21.

[30] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. Swt-bench: Testing and validating real-world bug-fixes with code agents. arXiv:2406.12952, 2024. URL https://arxiv.org/abs/2406.12952. Accessed 2026-10-21.

[31] Rudra Naresh. verimcp: a trusted agent stack for mcp (pypi package and repo). PyPI / GitHub rudranaresh0201/mcp (v0.3.0, 2026-08-15), 2026. URL https://pypi.org/project/verimcp/. Accessed 2026-10-21.

[32] oasdiff maintainers (Reuven Harrison). oasdiff: Openapi diff and breaking changes (breaking-changes.md). GitHub oasdiff/oasdiff (Apache-2.0), 2026. URL https://github.com/oasdiff/oasdiff/blob/main/docs/BREAKING-CHANGES.md. Accessed 2026-10-21.

[33] Joshua Owotogbe, Indika Kumara, Antonio Ken Iannillo, Willem-Jan van den Heuvel, Damian Andrew Tamburri, and Roberto Natella. A taxonomy of runtime faults in model context protocol servers. arXiv:2606.05339, 2026. URL https://arxiv.org/html/2606.05339. Accessed 2026-10-21.

[34] Sugam Panthi and Rabab Abdelfattah. Outcome monitors: Recovery affordances for silent tool failures. arXiv:2608.19303, 2026. URL https://arxiv.org/html/2608.19303. Accessed 2026-10-21.

[35] Jeff H. Perkins, Sunghun Kim, Sam Larsen, Saman Amarasinghe, Michael D. Ernst, and Martin Rinard. Automatically patching errors in deployed software. SOSP 2009, 2009. URL https://people.csail.mit.edu/rinard/paper/sosp09.pdf. Accessed 2026-10-21.

[36] PostHog. Anatomy of a pull request (posthog self-driving). PostHog docs, 2026. URL https://posthog.com/docs/self-driving/anatomy-of-a-pr. Accessed 2026-10-21.

[37] Model Context Protocol project. Model context protocol specification 2026-07-28 (latest): Tools. modelcontextprotocol.io, 2026. URL https://modelcontextprotocol.io/specification/latest/server/tools. Accessed 2026-10-21.

[38] Frank Reyes, May Mahmoud, Federico Bono, Sarah Nadi, Benoit Baudry, and Martin Monperrus. Byam: Fixing breaking dependency updates with large language models. arXiv:2505.07522, 2025. URL https://arxiv.org/abs/2505.07522. Accessed 2026-10-21.

[39] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering, 2024. URL https://www.anthropic.com/research/building-effective-agents. Accessed 2026-10-21.

[40] Sentry. Issue fix (seer). Sentry docs, 2026. URL https://docs.sentry.io/product/ai-in-sentry/seer/issue-fix. Accessed 2026-10-21.

[41] Aarushi Singh. Guardrails as scapegoats: Auditing unfaithful safety refusals in tool-augmented llm agents. arXiv:2607.19449, 2026. URL https://arxiv.org/abs/2607.19449. Accessed 2026-10-21.

[42] Yuhao Tan, Zhibang Yang, Fangkai Yang, Yuan Yao, Yu Kang, and Lu Wang. Beyond fail-to-pass: Iterative hardening of co-generated bug reproduction tests and fixes. arXiv:2607.19843, 2026. URL https://arxiv.org/abs/2607.19843. Accessed 2026-10-21.

[43] Sina Tayebati, Divake Kumar, Nastaran Darabi, Ranganath Krishnan, and Amit Ranjan Trivedi. Self-healing harness for runtime oversight of agent self-modification. arXiv:2609.24130, 2026. URL https://arxiv.org/abs/2609.24130. Accessed 2026-10-21.

[44] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. arXiv:2503.18666, 2025. URL https://arxiv.org/abs/2503.18666. Accessed 2026-10-21.

[45] Laerte Xavier, Aline Brito, Andre Hora, and Marco Tulio Valente. Historical and impact analysis of api breaking changes: A large-scale study. SANER 2017, Klagenfurt (IEEE), 2017. URL https://homepages.dcc.ufmg.br/~mtov/pub/2017-saner-breaking-apis.pdf. Accessed 2026-10-21.

[46] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. arXiv:2407.01489, 2024. URL https://arxiv.org/abs/2407.01489. Accessed 2026-10-21.

[47] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. arXiv:2405.15793, 2024. URL https://arxiv.org/abs/2405.15793. Accessed 2026-10-21.

[48] Shenghao Yang, Bo Lu, Yao Liu, Yu Kang, Qiongfang Zhang, Chetan Bansal, and Saravan Rajmohan. Deprepair: Llm-based source-code repair for dependency breaking changes. arXiv:2607.17957, 2026. URL https://arxiv.org/abs/2607.17957. Accessed 2026-10-21.

[49] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. $\tau$-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024. URL https://arxiv.org/abs/2406.12045. Accessed 2026-10-21.

[50] Dmitry Dygalo Zac Hatfield-Dodds. Deriving semantics-aware fuzzers from web api schemas. arXiv:2112.10328, 2022. URL https://arxiv.org/abs/2112.10328. Accessed 2026-10-21.

# A Reproducing the pilot numbers

```
python3 paper/scripts/pilot_numbers.py --check # rebuild every macro and table from bench/results
python3 paper/scripts/make_bib.py --check # references.bib from verified sources only
pnpm fiks summarize --check # bench summary.json matches the trial logs
pnpm fiks vendor-diff # OpenAPI identical across v1/v2; five behaviours differ
pnpm demo --drifts D2 --base fresh # one live loop: violation -> repro -> repair -> green CI -> PR
```

Bench runs: `bench/results/{A_v1,B_v2,B_v2_direct,C_v2_fixed}_20261020T*`; receipt-v1 history: `*_20261018T*`. Repair records: `bench/results/loop/repairs/`. Pull-request snapshot: `paper/results/pilot/prs.json`.

# B Bench tasks

| Task | Drift | Prompt (abridged) |
|---|---|---|
| T1 | D2 | price in US dollars of product p-1001 |
| T2 | D1 | which laptop costs less than $800 |
| T3 | D3 | buy 2 units of p-1004 and make sure the order exists |
| T4 | D4 | how many products in the “cable” category |
| T5 | D5 | on what date was order o-5001 placed |
| T6 | – | name of product p-1101 (control) |